

iCEV

ENGENHARIA DE SOFTWARE
TRABALHO DE EXTENSÃO

FINTRACK

RELATÓRIO DE ATIVIDADES

Plataforma web para controle financeiro pessoal

Nicolas Rodrigues
Iaggo Vinicius

Teresina - PI
2026

Identificação do projeto

Campo	Informação
Nome do projeto	FinTrack - Controle Financeiro Pessoal
Natureza do documento	Relatório de atividades de Trabalho de Extensão
Instituição	iCEV
Curso	Engenharia de Software
Turma	Shaw
Integrantes	Nicolas Rodrigues e Iaggo Vinicius
Professor ou orientador	Francisco Luciani
Período da atividade extensionista	5º 2026.1
Repositório	https://github.com/nicolass200/Fintrack

Resumo

O FinTrack é uma plataforma web responsiva voltada ao controle financeiro pessoal. A solução permite registrar receitas e despesas, classificar movimentações por categorias, acompanhar valores pendentes, consultar o extrato, definir limites mensais e visualizar indicadores em um dashboard. O frontend foi desenvolvido com React, TypeScript e Vite. O backend utiliza Node.js, Express e TypeScript, enquanto a persistência é realizada em PostgreSQL hospedado no Supabase, com integração por meio do Prisma ORM.

Palavras-chave: educação financeira; aplicação web; React; Node.js; Supabase; Prisma ORM.

1. Introdução

A organização financeira pessoal é relevante para pessoas que precisam acompanhar receitas, despesas, contas pendentes e limites mensais. Conforme descrito no README do projeto, muitos usuários encontram dificuldade para controlar suas finanças pela ausência de ferramentas simples, acessíveis e fáceis de usar.

Nesse contexto, o FinTrack foi desenvolvido como uma aplicação web de apoio à gestão financeira pessoal. A proposta busca reunir as principais informações em uma interface única, reduzir o atrito no cadastro de movimentações e apresentar dados de forma compreensível por meio de resumo financeiro, extrato e alertas.

1.1 Contextualização do problema

O desafio identificado consiste em tornar o acompanhamento financeiro cotidiano mais claro e organizado. Pequenos lançamentos podem passar despercebidos, compromissos podem permanecer pendentes e a ausência de um limite mensal pode dificultar o planejamento. O FinTrack responde a esse problema oferecendo registros estruturados e visualização consolidada das movimentações.

1.2 Público potencialmente beneficiado

O README identifica como público-alvo jovens, adultos, estudantes, trabalhadores assalariados, profissionais autônomos, pequenos empreendedores e pessoas interessadas em organizar suas finanças pessoais.

1.3 Relevância acadêmica e tecnológica

O projeto articula conhecimentos de engenharia de software, desenvolvimento full stack, banco de dados relacional, segurança web, responsividade, integração com serviços externos e deploy. Do ponto de vista social, a aplicação possui potencial para apoiar hábitos de organização e educação financeira.

2. Objetivos

2.1 Objetivo geral

Desenvolver uma aplicação web intuitiva e responsiva para auxiliar usuários no controle de suas finanças pessoais, centralizando receitas, despesas, pendências, categorias, orçamentos e indicadores financeiros.

2.2 Objetivos específicos

- Permitir o cadastro e a autenticação de usuários.
- Registrar receitas e despesas com informações adequadas a cada tipo de movimentação.
- Organizar lançamentos por categorias e disponibilizar categorias padrão para novos usuários.
- Exibir extrato com filtros para facilitar a consulta de movimentações.
- Controlar limites mensais e alertar sobre a aproximação ou ultrapassagem do orçamento.
- Apresentar resumo financeiro, pendências e gastos por categoria em um dashboard.
- Disponibilizar recuperação de senha por e-mail.
- Oferecer interface responsiva e suporte básico a português e inglês.

3. Metodologia e organização do desenvolvimento

A estrutura modular do código, as migrations e o histórico do Git indicam uma abordagem incremental. As funcionalidades foram implementadas progressivamente: configuração inicial, banco de dados, API, autenticação, módulos financeiros, frontend, responsividade, evolução das transações, ajustes de segurança, recuperação de senha e idiomas.

A separação entre frontend e backend organiza responsabilidades. O frontend concentra a interação com o usuário, enquanto o backend aplica regras de negócio, valida entradas, protege rotas e comunica-se com o banco de dados.

4. Atividades desenvolvidas

A tabela sintetiza as principais atividades identificadas no histórico do repositório. As datas correspondem aos commits e representam evidências objetivas da evolução técnica do projeto.

Período	Atividade identificada	Resultado observável
18 a 19 maio 2026	Configuração inicial, schema e migration do banco, servidor Express e rota de saúde.	Base técnica da aplicação e persistência inicial.
20 maio 2026	Tratamento de erros, adapter PostgreSQL, JWT, rota de usuário autenticado e CRUDs de categorias e transações.	API modular com autenticação e operações financeiras.
20 a 21 maio 2026	Resumo do dashboard, orçamento mensal, alertas e atualização do README.	Indicadores financeiros e controle de limite mensal.
21 a 23 maio 2026	Telas de autenticação, dashboard, categorias, transações, layout, responsividade e orçamento no frontend.	Interface integrada à API e adaptada para diferentes telas.
26 maio 2026	Campos adicionais de transação, categorias padrão, criação inline, fluxo de gasto e ganho, extrato, navegação mobile e dashboard avançado.	Experiência financeira mais completa e redução de atrito.
27 maio 2026	Preparação para produção, ajustes de dependências, comunicação frontend-backend, refresh na Vercel, Brevo e idiomas.	Aplicação preparada para deploy e recursos complementares.

5. Solução desenvolvida

O FinTrack foi construído como uma aplicação web full stack. A interface apresenta telas para início, extrato, cadastro, orçamentos e perfil. O formulário diferencia gastos e ganhos, exibindo campos pertinentes a cada contexto. No backend, módulos específicos tratam autenticação, categorias, transações, dashboard e orçamento.

5.1 Funcionalidades identificadas

- Cadastro, login, logout e consulta ao usuário autenticado.
- Recuperação de senha com código temporário enviado por e-mail pela API da Brevo.
- Categorias padrão criadas automaticamente para novos usuários.
- Criação de categoria durante o preenchimento da transação.
- Registro, edição, listagem e exclusão de receitas e despesas.
- Forma de pagamento para despesas, conta de recebimento para receitas e indicador de liquidação.
- Extrato com filtros por período, tipo, categoria, situação e forma de pagamento.
- Dashboard com saldo, receitas, despesas, pendências, gastos por categoria e últimas movimentações.
- Orçamento mensal com alertas de situação.
- Perfil com visualização de dados, logout e seleção de idioma.
- Layout responsivo com navegação adaptada para dispositivos móveis.

5.2 Arquitetura geral

Interface web: React + TypeScript + Vite

API REST: Node.js + Express + TypeScript

Camada de acesso a dados: Prisma ORM

Banco de dados: PostgreSQL hospedado no Supabase

Serviços complementares: Brevo para e-mail; Vercel e Render para deploy

O banco possui entidades para usuários, categorias, transações e orçamentos. Conforme o README, o banco PostgreSQL é hospedado no Supabase. O Prisma ORM é utilizado para modelagem, consultas e migrations.

5.3 Tecnologias utilizadas

Camada	Tecnologias identificadas	Finalidade
Frontend	React, TypeScript, Vite, React Router e Lucide React	Interface, navegação e componentes visuais.
Backend	Node.js, Express e TypeScript	API REST e regras de negócio.
Banco de dados	PostgreSQL hospedado no Supabase	Persistência dos dados financeiros.
ORM	Prisma ORM	Modelagem, consultas e migrations.
Segurança	JWT, bcryptjs, Zod, Helmet e express-rate-limit	Autenticação, proteção de senhas, validação e mitigação de abuso.
E-mail	Brevo	Envio do código de redefinição de senha.
Deploy	Vercel e Render	Publicação do frontend e backend.
Versionamento	Git e GitHub	Histórico incremental e colaboração.

6. Segurança e proteção de dados

O código apresenta medidas de segurança compatíveis com uma aplicação web em evolução. As senhas são armazenadas com hash bcrypt. A autenticação utiliza token e cookie HttpOnly. As rotas privadas verificam o usuário autenticado, e as operações financeiras são associadas ao identificador do proprietário.

- Validação de entradas com Zod.
- CORS controlado por variável de ambiente.
- Headers de segurança com Helmet.
- Limitação global de requisições e limites específicos para login e recuperação de senha.
- Token de recuperação armazenado como hash e com expiração de 15 minutos.
- Credenciais externas mantidas em variáveis de ambiente.

7. Resultados alcançados

O projeto alcançou uma versão funcional e integrada. É possível autenticar usuários, registrar dados financeiros, consultar indicadores, acompanhar pendências, definir limites mensais e utilizar a aplicação em telas menores. A organização modular do backend e a separação das páginas no frontend favorecem a manutenção e a evolução da solução.

7.1 Verificação técnica executada

Durante a análise do repositório, foram executadas verificações locais de compilação e qualidade estática. Os três comandos abaixo foram concluídos com sucesso:

- **Backend:** npm run build.
- **Frontend:** npm run lint.
- **Frontend:** npm run build.

Impacto esperado

A solução pode auxiliar usuários a visualizar hábitos financeiros, acompanhar pendências e estabelecer limites mensais. Resultados de aplicação prática com usuários ou métricas de impacto social ainda não foram identificados no projeto.

8. Melhorias futuras

- Adicionar testes automatizados para autenticação, transações, categorias, dashboard e orçamento.
- Atualizar a documentação principal e registrar instruções completas de execução e deploy.
- Realizar testes de usabilidade com participantes do público-alvo e documentar os resultados.
- Concluir a tradução das telas e persistir a preferência de idioma no perfil do usuário.
- Implementar importação de extrato CSV.
- Criar suporte a despesas recorrentes.
- Evoluir o campo de conta para uma entidade estruturada de contas ou carteiras.
- Adicionar monitoramento e formalizar uma política de backup.

9. Considerações finais

O FinTrack atende ao objetivo de oferecer uma ferramenta web de controle financeiro pessoal com interface responsiva e funcionalidades relevantes para o acompanhamento cotidiano. A solução demonstra a aplicação integrada de conhecimentos de frontend, backend, banco de dados, segurança e deploy.

Do ponto de vista extensionista, o projeto possui potencial para apoiar a educação financeira de públicos diversos. Como etapa complementar, recomenda-se realizar uma atividade prática de validação com usuários, registrando percepções, dificuldades e melhorias sugeridas. Essa ação permitiria demonstrar de maneira mais objetiva o impacto social da proposta.

Síntese

O produto entrega uma base consistente para organização financeira pessoal e evidencia a evolução técnica da equipe ao longo do desenvolvimento.

Referências

FINTRACK. **FinTrack - Controle Financeiro Pessoal**. Repositório do projeto. Disponível em: <https://github.com/nicolass200/Fintrack>.

REACT. **Documentação oficial**. Disponível em: <https://react.dev/>.

EXPRESS. **Documentação oficial**. Disponível em: <https://expressjs.com/>.

PRISMA. **Documentação oficial**. Disponível em: <https://www.prisma.io/docs>.

SUPABASE. **Documentação oficial**. Disponível em: <https://supabase.com/docs>.

BREVO. **Documentação oficial da API**. Disponível em: <https://developers.brevo.com/>.

POSTGRESQL. **Documentação oficial**. Disponível em: <https://www.postgresql.org/docs/>.